

RELATÓRIO DE SESSÃO

Agendamento de Tarefas com `/loop`

Registro do trabalho realizado nesta sessão: envio de material do curso, implementação e teste de três automações de exemplo com o comando `/loop`, e um incidente real de sincronização resolvido no caminho.

3 exemplos testados

3 commits enviados

0 código em produção

Projeto: Formação Claude Code 2026 — Claude Code Avançado

Repositório: github.com/fernandojsmelo/FormacaoClaudeCode2026

Autor: Fernando José da Silva Melo

Data: 9 de setembro de 2026

Assistente: Claude Sonnet 5 (Claude Code)

1 Resumo executivo

A sessão teve dois focos: primeiro, subir para o repositório o material sobre o comando `/loop` (PDF do curso); segundo, transformar a dúvida "como eu poderia usar agendamento com o `/loop` nesse projeto?" em três automações de exemplo **reais, implementadas e testadas** — sem instalar nada em produção, conforme pedido.

No caminho, um incidente real de sincronização do Dropbox reorganizou pastas sozinho por duas vezes; o problema foi diagnosticado (processo do Dropbox ativo na máquina) e corrigido. Por fim, a estrutura final foi ajustada a pedido: a pasta de exemplos ficou como subpasta de `Loop/`, no mesmo padrão usado por `Hooks/` e `SubAgents/`.

2 Linha do tempo

Commit	O que foi feito
b205fa4	Envio do material em PDF sobre o comando <code>/Loop</code> (<code>Loop_Tarefas_Agendadas.pdf</code>) para o repositório.
2d83c08	Criação dos 3 slash commands de exemplo com testes documentados, e correção de um nome de pasta com aspas/barra mal formadas.
98b4110	Reorganização: pasta <code>Agendamento com Loop</code> movida para dentro de <code>Loop/</code> , seguindo o padrão das demais pastas de tema do curso.

3 O comando `/loop`

Roda um prompt ou slash command repetidamente na mesma sessão do Claude Code:

```
/loop [intervalo] <prompt ou slash command>
```

- **Com intervalo** (ex.: `/loop 20m /verificar-novos-materiais`) — cronograma fixo, em minutos ou horas.
- **Sem intervalo** (ex.: `/loop /checar-padrao-pastas`) — modo dinâmico: o próprio modelo decide quando reagendar a próxima rodada, conforme o que encontrou.

Limitação importante: só existe enquanto o terminal está aberto — não é um cron do sistema. Para algo que precise sobreviver ao fechamento da sessão, o caminho é a skill `schedule` (agentes com cron real na nuvem), não o `/loop`.

4 Os três exemplos implementados

Cada exemplo é um slash command (arquivo `.md` que viveria em `.claude/commands/`), testado em modo headless (`claude -p`) contra um **repositório git temporário e isolado** — nunca contra este projeto. Nenhum foi instalado aqui, e nenhum `/loop` ficou rodando de fato.

1. Commit automático de materiais novos

AÇÃO REVERSÍVEL

Detecta arquivos não versionados e cria um commit local descritivo — nunca dá `git push`. `allowed-tools` restrito a `git status/add/commit/log`, então fisicamente não consegue enviar nada ao remoto.

Teste 1 (arquivo novo): "Commit criado localmente (eaea704): adicionou Módulo-Teste/aula-nova.pdf (...). Nenhum push foi feito."

Teste 2 (nada novo, idempotência): "Nada para commitar."

2. Atualização do índice no README

EDIÇÃO CIRÚRGICA

Escaneia as pastas de nível superior e atualiza a lista de conteúdo entre marcadores `<!-- INICIO-INDICE -->` / `<!-- FIM-INDICE -->`, sem tocar no resto do arquivo.

Teste 1 (índice vazio): preencheu corretamente com Hooks, Loop e SubAgents em ordem alfabética.

Teste 2 (idempotência): pegadinha real — a resposta saiu parcialmente em inglês na primeira tentativa. Ajustei o comando para exigir português; no reteste, com uma 4ª pasta criada, atualizou só o item novo e preservou o resto do arquivo.

3. Checagem de padrão de pastas

SOMENTE LEITURA

Confere se as pastas de exemplo seguem o padrão `exemplo-N-descricao` com `README.md`, e relata divergências — sem corrigir nada. `allowed-tools` restrito a `find` e `Read`: não tem como editar nada mesmo que tentasse.

Teste (pasta fora do padrão de propósito): identificou corretamente `teste-rapido/` como divergente (nome fora do padrão + README ausente), sem falso positivo nas pastas conformes.

5 Incidente: interferência do Dropbox

Durante a sessão, a pasta `Agendamento com Loop` foi movida sozinha para dentro de `Loop/` por duas vezes, sem nenhum comando meu fazendo isso — e uma pasta pré-existente (`Compreendendo o Comando "/Loop"`) apareceu com o nome sanitizado, perdendo aspas e barra.

Diagnóstico

A investigação (`ps aux`) encontrou o processo `/home/fernandomelo/.dropbox-dist/.../dropbox` ativo — o projeto está dentro de uma pasta sincronizada pelo Dropbox. A causa mais provável: um `mkdir` meu, executado por engano com o diretório de trabalho errado no início dos testes, criou uma pasta vazia em `Loop/Agendamento com Loop`. Mesmo depois de eu apagar essa pasta vazia localmente, o mecanismo de sincronização/detecção de "arquivo movido" do Dropbox parece ter guardado esse caminho e reconciliado criações seguintes com o mesmo nome de volta para lá.

Resolução

Conferido por hash (`md5sum`) que o conteúdo nunca foi corrompido — só a localização mudou. A pasta foi movida de volta manualmente, o `git status` confirmado como estável, e o achado foi reportado ao usuário antes de qualquer novo commit. No fim, o usuário decidiu manter a pasta dentro de `Loop/` de propósito — o que virou a estrutura final do commit [98b4110](#).

6 Estrutura final em `Loop/`

```
Loop/
├── Compreendendo o Comando loop/
│   └── Loop_Tarefas_Agendadas.pdf
└── Agendamento com Loop/
    ├── README.md
    ├── exemplo-1-commit-automatico-de-materiais/
    │   ├── README.md
    │   └── verificar-novos-materiais.md
    ├── exemplo-2-atualizar-indice-readme/
    │   ├── README.md
    │   └── atualizar-indice-readme.md
    └── exemplo-3-checagem-padrao-pastas/
        ├── README.md
        └── checar-padrao-pastas.md
```

7 Lições registradas

- **Idempotência é pré-requisito para `/loop`** : os três comandos foram testados duas vezes de propósito (com e sem novidade) para garantir que rodar em loop sem supervisão não gera ruído (commits vazios, edições redundantes).
- **`allowed-tools` como rede de segurança**: cada comando só recebeu as ferramentas estritamente necessárias — o de commit não tem `push` , o de relatório não tem `Edit` . A restrição de permissão, não só a instrução em texto, é o que impede a ação indevida.
- **Idioma da resposta precisa ser explícito**: um comando sem instrução de idioma pode responder em inglês numa rodada "sem novidade" — vale fixar isso no prompt do comando, não só confiar em preferências gerais da sessão.
- **Ambiente sincronizado exige atenção redobrada**: um projeto dentro de uma pasta do Dropbox pode sofrer reorganizações automáticas de arquivos durante uma sessão de trabalho — vale checar `git status` com calma antes de assumir que um commit reflete o estado real do disco.

Relatório gerado automaticamente ao final da sessão — Claude Code (Sonnet 5)